

The Pareidolia Paradox

A Winning Roadmap for the Lunar Terrain Classification Challenge

Prepared for Aryan | Competition window: 1 Sep - 21 Sep 2026
Task: binary image classification | Metric: Balanced Accuracy

This document lays out how to think about the problem, why the obvious approach is a trap, and a concrete step-by-step build plan - including a few ideas specifically shaped around the physics of this dataset that most competitors won't bother with.

1. What This Competition Is Really Asking

Strip away the framing and the task is simple: for every 256x256 grayscale image of a patch of lunar surface, decide whether the dominant feature is a **depression** (a crater, hole, or dent - Class 0) or a **raised feature** (a mound, hill, rock, or boulder - Class 1). Along with every image you're given one extra number: **sun_azimuth_angle**, the compass direction (in the image's own frame) that sunlight is coming from.

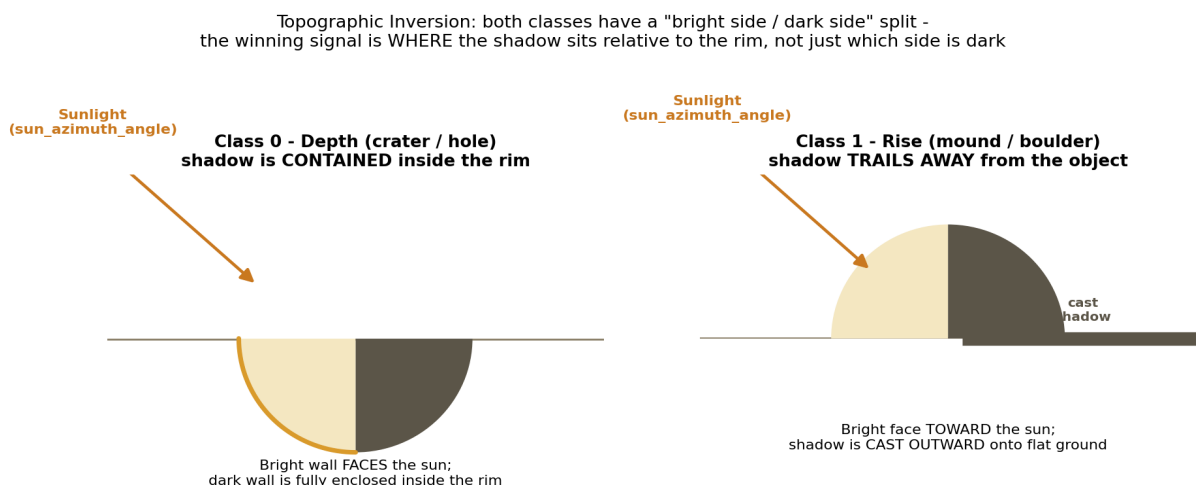
That one number is the entire reason this competition exists. On the Moon there is no atmosphere to scatter light and soften shadows, so the appearance of terrain is almost entirely a function of shading - how directly a surface faces the sun. A crater lit from the north-west can produce almost the exact same pattern of bright and dark pixels as a boulder lit from the south-east. Human vision has a built-in assumption that light comes from above, which is why the same photo can flip between looking like a pit and a bump depending on which way you tilt your head - the classic "crater/dome illusion." A naive image classifier inherits the same confusion. That's the paradox in the name.

Practically, this means **sun_azimuth_angle is not a minor auxiliary column - it is as important as the pixels themselves**. Any approach that treats it as an afterthought (or ignores it) is giving away the single biggest advantage available in this dataset.

One more detail matters for how you optimize: the metric is **Balanced Accuracy**, the average of the recall on Class 0 and the recall on Class 1. A model that is great at spotting craters but weak on boulders will score worse than the raw pixel-accuracy would suggest. If the training set is even mildly imbalanced, this changes how you should weight your loss function and where you set your decision threshold - both covered in Section 4.

2. The Physics You Can Exploit (and the Trap Hidden in It)

Surface brightness under directional lighting follows a simple rule: slopes tilted toward the sun look bright, slopes tilted away look dark. That single rule is what makes topographic inversion possible - but it's also what makes a purely "which side is dark" strategy fail, which is exactly why the organizers warn against "relying solely on visual shadow patterns."



Both classes show a bright/dark split relative to the sun. The distinguishing signal is the **SHAPE** and **LOCATION** of the dark region relative to the bright one - contained within a rim (crater) versus trailing outward across flat ground (mound/boulder).

Both a crater and a mound produce a bright half and a dark half once you know the sun's direction - so a model (or a hand-written rule) that only measures "which side is brighter" is measuring something that correlates with azimuth for *both* classes and therefore carries very little information about which class it's looking at. This is almost certainly why simple shadow-direction heuristics plateau well below the top of the leaderboard, and why the eval set likely includes cases designed to punish that shortcut (grazing light angles, small or ambiguous features, noisy patches).

The real, generalizable signal is geometric: a crater's shadow is a crescent **enclosed inside a rim**, while a boulder or mound's shadow is a wedge that **trails outward onto the surrounding flat ground**. That's a statement about shape, not just average brightness - which is precisely the kind of pattern a convolutional network with a reasonable receptive field is good at picking up, provided you set the problem up so it doesn't have to also learn to undo 360 degrees of rotation on top of it.

3. Five Ideas That Will Separate You From the Leaderboard Pack

These are the ideas most competitors will skip because they take an extra day to build. They're listed here in priority order - if time runs short, implement from the top down.

Idea	Why it wins	Effort
1. Azimuth canonicalization	Rotate every image so the sun always comes from the same reference direction before it reaches the model. This removes an entire axis of variation (absolute orientation) the network would otherwise have to learn to ignore.	Low
2. Physics-consistent augmentation	Rotating an image by some angle also rotates its true sun_azimuth_angle by that same angle. This gives you free, physically valid new (image, azimuth) training pairs from every real image - effectively multiplying your labeled dataset at no data-collection cost.	Low
3. Synthetic pretraining	Procedurally render toy craters and mounds under a Lambertian lighting model with random azimuth, elevation, and surface texture/noise. You can generate tens of thousands of perfectly labeled images for free and pretrain a backbone on the exact physics before fine-tuning on the real 7,854 images.	Medium
4. Dual-signal architecture	Feed the network both the raw grayscale image and an explicit "directional gradient" map (a Sobel-derived channel aligned with the known sun direction). This hands the model the shading physics directly instead of making it rediscover Sobel filters from scratch on ~8,000 images.	Medium
5. Azimuth-stratified error analysis + tuned ensembling	Break validation performance down by azimuth bucket to find the lighting angles where the model actually struggles (usually near-grazing angles), target those with extra augmentation, then ensemble multiple models and tune the decision threshold directly for Balanced Accuracy rather than accuracy.	Medium

4. Step-by-Step Build Plan

4.1 Exploratory Data Analysis (don't skip this)

- Check class balance in `train_metadata.csv` - this tells you whether to weight the loss and hints at what Balanced Accuracy will actually reward.
- Plot the distribution of `sun_azimuth_angle` for both classes, and compare the train vs test metadata distributions - if azimuth ranges differ between train and test, your validation split needs to account for that.
- Manually inspect ~20 images per class at a range of azimuth values. This is also how you'll reverse-engineer the exact azimuth convention (0 degrees = which direction? clockwise or counter-clockwise? measured from the top of the image?) - the metadata description won't tell you this, so confirm it visually before you build any rotation logic.
- Check image quality: any corrupted files, unusual bit depths, or duplicate images between classes.

4.2 Canonicalize Illumination Direction

- Once you've confirmed the azimuth convention, rotate every training and test image so the sun direction always points the same way (e.g. always "from the left"). Use bicubic interpolation and reflect-padding to minimize corner artifacts, since these terrain features are usually centered in the frame.
- Keep the un-rotated image available too - some architectures (see 4.5) benefit from seeing both the canonical and the raw view.
- Sanity-check the rotation by re-plotting the same 20 images from 4.1 after canonicalization - the bright side should now be visually consistent across all of them.

4.3 Physics-Consistent Data Augmentation

- Standard augmentation (random rotation, flips) breaks this dataset unless you update the azimuth to match. The fix: whenever you rotate an image by some angle during training, add that same angle to its azimuth value (mod 360) before it's used downstream. Horizontal and vertical flips need their own reflection formula for the azimuth - work this out empirically against a couple of test images rather than trusting a formula blindly, since it depends on your coordinate convention.
- This turns every one of your 7,854 training images into an effectively unlimited supply of valid (image, azimuth, label) triples, and it directly teaches the model that what matters is the *relationship* between shading and azimuth, not absolute orientation.
- Encode azimuth as $(\sin(\text{angle}), \cos(\text{angle}))$ rather than a raw 0-360 number wherever it's fed to a model - otherwise the model sees 1 degree and 359 degrees as far apart when they're almost identical.

4.4 Hand-Crafted Physics Features (for a complementary model)

Alongside the deep model, build a small feature set that captures the geometry directly - useful both as an ensemble partner and for debugging what the network has learned:

- Directional gradient map: a Sobel-based derivative computed along the sun-azimuth direction (after canonicalization this is just a plain horizontal Sobel filter).
- Shadow-region shape: threshold the darkest pixels, find the resulting blob, and measure its position, elongation, and compactness relative to the image center - an enclosed crescent (crater) looks very

different from a wedge trailing off-frame (mound).

- Radial intensity profile from the image center outward, and basic texture measures (local variance / Laplacian energy) for roughness.
- Feed these into a gradient-boosted tree model (LightGBM or XGBoost) as a second, independent opinion for the final ensemble.

4.5 Model Architecture

- Backbone: a lightweight ImageNet-pretrained CNN (ResNet18/34 or EfficientNet-B0/B1) is a better starting point than something huge - with ~7,854 images, a large backbone will overfit before it learns anything a smaller one can't. Replicate the grayscale channel to 3 channels so pretrained weights stay usable.
- Fusion of azimuth: after global average pooling, concatenate the CNN's feature vector with the sin/cos-encoded azimuth (and, optionally, the hand-crafted features from 4.4) and pass the combined vector through a small MLP head with dropout before the final classification layer.
- Dual-channel input (recommended): stack the canonicalized grayscale image and its directional gradient map as two input channels. This bakes the shading physics directly into what the network sees, rather than hoping late fusion learns to weight it correctly.
- Train several architecturally different models (e.g. one CNN on canonicalized single-channel input, one on the dual-channel input, one gradient-boosted tree on hand-crafted features) - diversity here is what makes the ensemble in 4.9 actually help.

4.6 Synthetic Pretraining (the highest-leverage "extra" idea)

With only 7,854 real images, the model has limited exposure to the full range of lighting angles and terrain shapes. Because the underlying physics is well understood (Lambertian shading of a bump or a pit), you can manufacture as much correctly-labeled training data as you want:

- Procedurally generate simple height maps: paraboloid or Gaussian "bumps" for Class 1 and "pits" for Class 0, with randomized size, depth/height, and position.
- Render each height map under a Lambertian shading model with a randomly sampled azimuth (and a plausible elevation angle), then add regolith-like texture and sensor noise so the renders don't look artificially clean.
- Pretrain your backbone on tens of thousands of these synthetic pairs, then fine-tune on the real 7,854 images. This gives the network far more exposure to the azimuth-shading relationship than the real dataset alone can provide, and is a genuinely uncommon move at most hackathons.
- Validate that this actually helps (compare against a non-pretrained baseline on your held-out fold) before committing to it for the final model - synthetic-to-real gaps are real, and this is exactly what cross-validation in 4.8 is for.

4.7 Training Regime

- Loss: binary cross-entropy with class weighting (or focal loss if 4.1 revealed real imbalance).
- Optimizer: AdamW with a cosine-annealed learning rate and a short warmup; mixed precision to speed up iteration.
- Regularization: dropout in the classification head, modest weight decay, and mild label smoothing. Be cautious with MixUp/CutMix here - blending two images with different azimuths creates a physically inconsistent training example, so if you try it, don't mix the azimuth metadata naively.

4.8 Validation Strategy

- Use stratified k-fold (5 folds is a good default) stratified by label. Split by unique base image *before* augmentation, so augmented copies of the same source image never end up split across train and validation - that would leak information and make your local score overly optimistic.
- Also check the azimuth distribution is reasonably similar across folds, so no fold is accidentally testing on lighting angles the model barely saw in training.
- Track Balanced Accuracy directly during validation, not plain accuracy - they can diverge noticeably if the classes are imbalanced.

4.9 Ensembling and Test-Time Augmentation

- Average predictions across your k folds and across your architecturally different models (4.5). A stacked meta-learner (simple logistic regression on out-of-fold predictions) typically beats a flat average once you have 3+ diverse models.
- Test-time augmentation: predict on the original image plus a horizontal and vertical flip (with correctly recomputed azimuth for each), and average the resulting probabilities.
- Tune the final decision threshold on out-of-fold predictions to directly maximize Balanced Accuracy - the default 0.5 cutoff is rarely optimal once you're this deep into optimization.

4.10 Final Submission

- Retrain your final chosen recipe on 100% of the training data (no held-out fold) once the approach is locked in, using the hyperparameters validated in cross-validation.
- Generate predictions for all 2,000 evaluation images and write a CSV with exactly the columns `image_id,label` - double check the `image_id` strings match the test metadata exactly (including file extension), that every one of the 2,000 rows is present, and that labels are integers (0 or 1), not floats or strings.

5. Tooling Cheat Sheet

Purpose	Suggested tools
Deep learning	PyTorch + timm (for pretrained ResNet/EfficientNet backbones)
Image augmentation	albumentations, with custom transforms for azimuth-aware rotation/flips
Classical CV features	OpenCV (Sobel, connected components, radial profiles)
Tree-based model	LightGBM or XGBoost on hand-crafted features
Experiment tracking	A simple CSV/spreadsheet log of fold scores per experiment is enough at this scale
Synthetic data rendering	NumPy/SciPy for height maps + a basic Lambertian shading function (no special renderer needed)

6. Pitfalls That Will Quietly Sink Your Score

- **Treating azimuth as linear instead of circular.** Feeding the raw 0-360 number into a model makes it think 1 degree and 359 degrees are opposite when they're nearly identical. Always use sin/cos encoding.
- **Leaking augmented copies across train/validation splits.** Split by base image first, augment after.
- **Rotating images without rotating the azimuth to match.** This silently breaks the physical consistency of your dataset and can hurt more than it helps.
- **Trusting a raw "which side is brighter" heuristic.** As shown in Section 2, this signal is shared by both classes and is exactly the shortcut the organizers are warning against.
- **Optimizing for plain accuracy instead of Balanced Accuracy.** Tune your threshold and loss weighting against the actual competition metric.
- **Submission formatting slip-ups.** Wrong column names, missing rows, float labels instead of integers, or mismatched image_id strings are avoidable losses - verify against the exact example format before your final submission.

7. Your Day-by-Day Roadmap (5 Sep - 21 Sep)

The competition opened 1 September; starting today (5 September) still leaves 17 days. Here's how to spend them.

Dates	Focus	Deliverable
Sep 5 - 6	Setup, EDA, confirm the azimuth convention empirically, train a quick baseline CNN with no azimuth handling at all.	First submission on the board
Sep 7 - 9	Build and verify the azimuth-canonicalization pipeline; implement physics-consistent rotation/flip augmentation; retrain the baseline with these in place.	Submission v2 (canonicalized + augmented)
Sep 10 - 12	Build hand-crafted physics features and a LightGBM model; build the dual-channel CNN (image + directional gradient map); set up proper k-fold cross-validation.	Submission v3 + working CV harness
Sep 13 - 15	Build the synthetic crater/mound renderer; pretrain a backbone on it; fine-tune on real data and compare against non-pretrained results in CV.	Decision: keep or drop synthetic pretraining
Sep 16 - 17	Error analysis broken down by azimuth bucket; target weak angle ranges with extra augmentation; hyperparameter tuning.	Tuned individual models
Sep 18 - 19	Ensemble all models (CNN variants + tree model) with a stacked meta-learner; add test-time augmentation; tune the decision threshold for Balanced Accuracy.	Near-final ensemble pipeline
Sep 20 - 21	Retrain the final recipe on 100% of training data; generate predictions; run every sanity check in Section 6 on the submission file; keep buffer time for last-minute fixes.	Final submission

8. Final Submission Checklist

- ☐ CSV has exactly the header `image_id,label`
- ☐ All 2,000 evaluation images are present, no duplicates, no missing rows
- ☐ `image_id` strings exactly match `test_metadata.csv` (including file extension)
- ☐ `label` column contains only integers 0 or 1 (no floats, no strings, no NaNs)
- ☐ Final model was trained on 100% of available training data, not just one CV fold
- ☐ Decision threshold was tuned for Balanced Accuracy, not left at a default 0.5
- ☐ You kept a note of which experiment produced your best local CV score, in case you need to reproduce or explain your approach

Good luck with it, Aryan - the physics-grounded ideas here (canonicalization, matched augmentation, synthetic pretraining) are exactly the kind of thing that separates a top-of-leaderboard solution from a merely correct one.